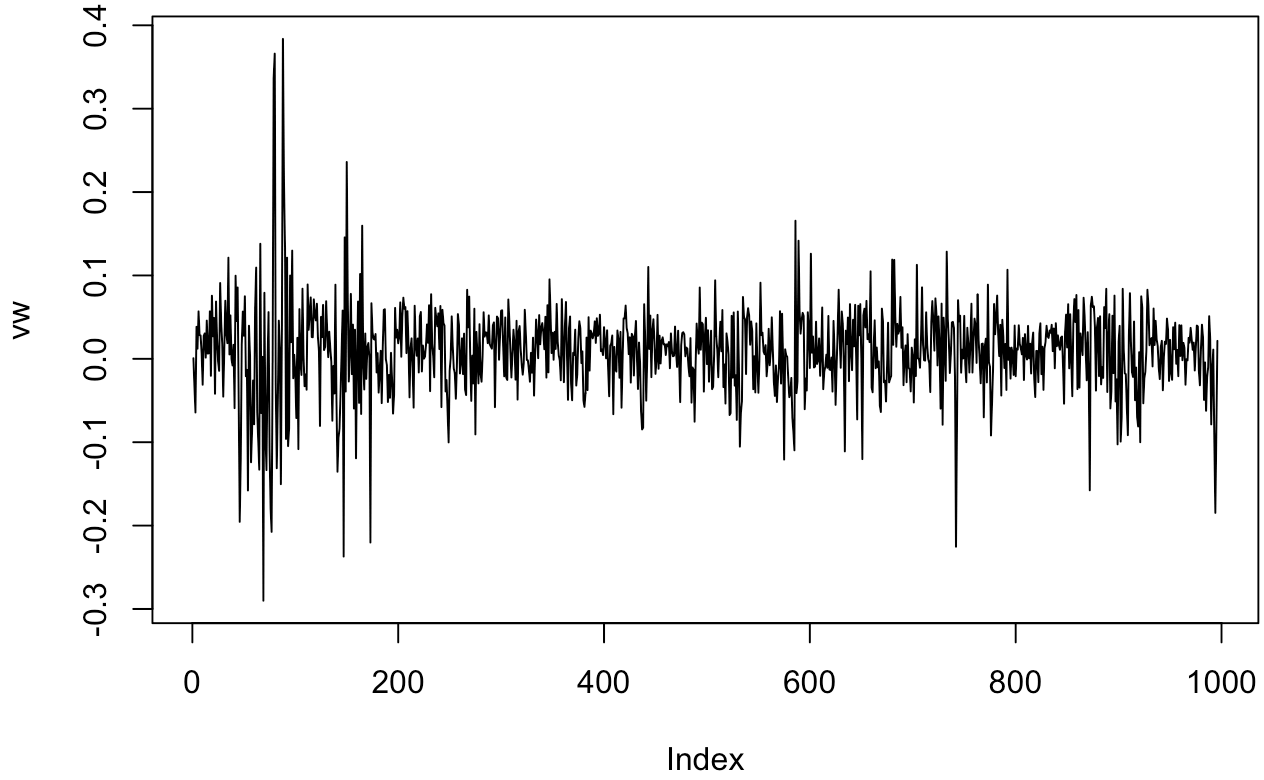
Lecture 06. ARMA models: model building and diagnostics

1. Estimation: AR Models.

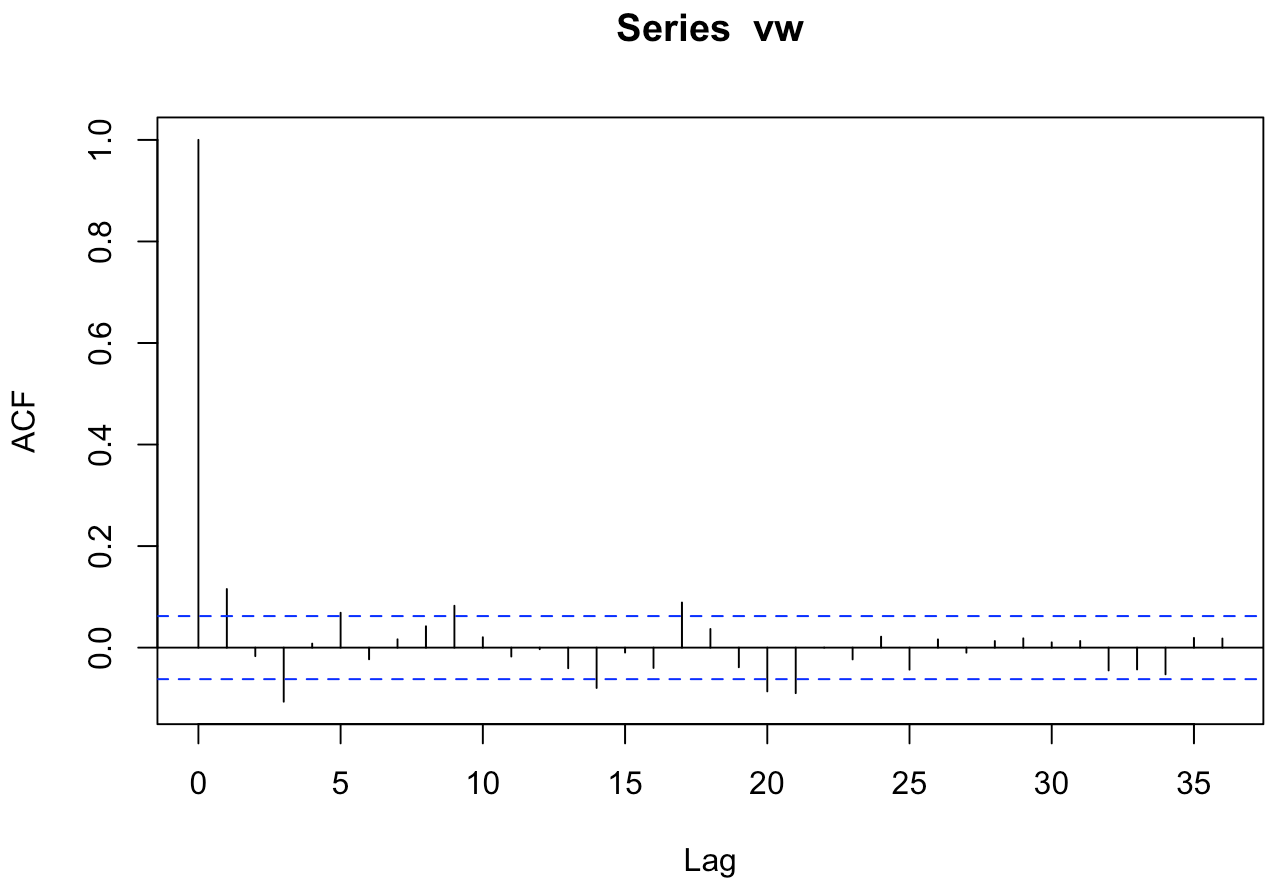
```
library(fBasics) # Load the package fBasics.
library(tseries)
library(astsa)

###################### monthly simple returns of the value weighted index from January 1
926 to December 2008
da=read.table("m-ibm3dx2608.txt",header=T)
da[1,]
```

```
##       date    ibmrtn    vwrtn    ewrtn    sprtn
## 1 19260130 -0.010381 0.000724 0.023174 0.022472
```

```
vw=da[,3] ## value weighted index returns
plot(vw,type="l")
```



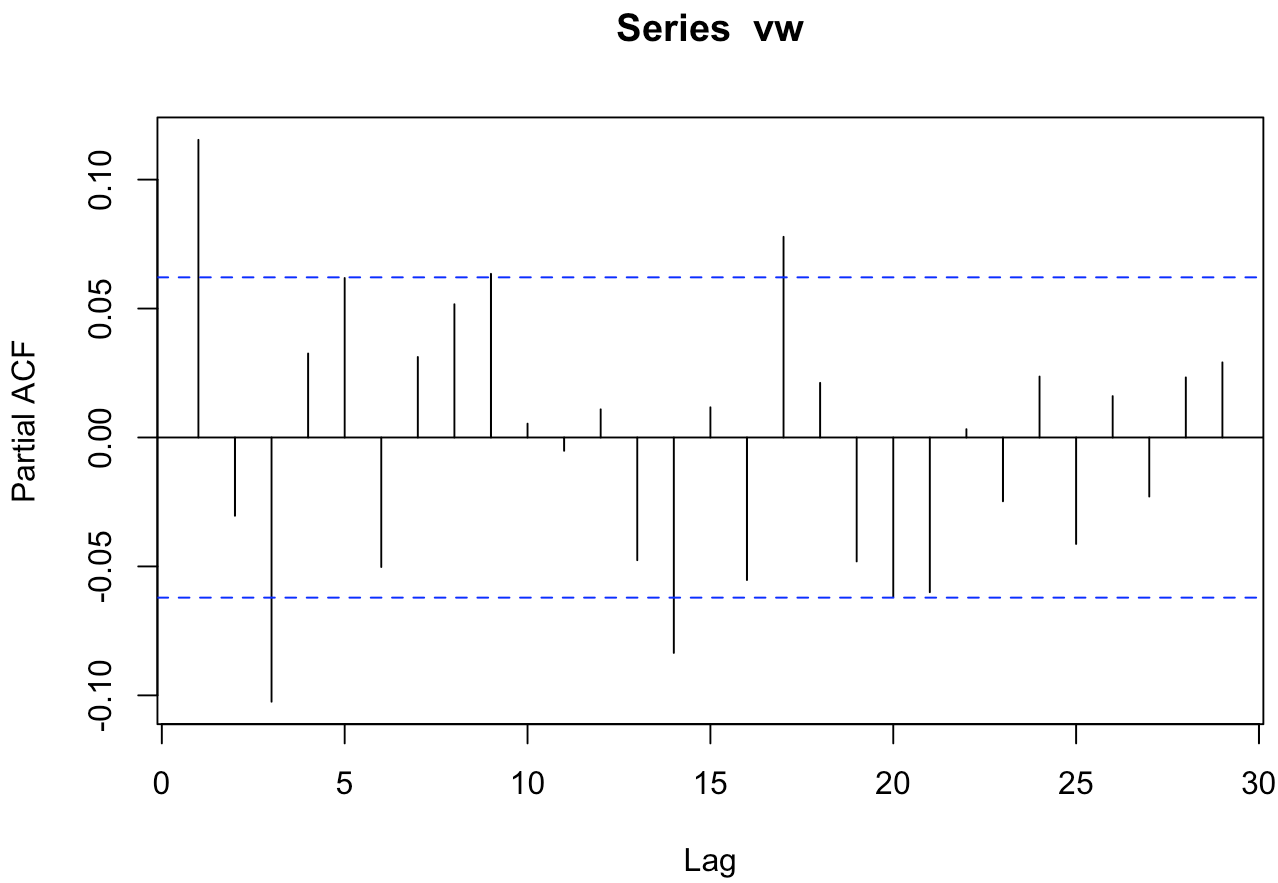

```
acf(vw,lag=36)
```




```
Box.test(vw,lag=5,type="Ljung")
```

```
##
##  Box-Ljung test
##
## data:  vw
## X-squared = 29.711, df = 5, p-value = 1.681e-05
```

```
pacf(vw)
```



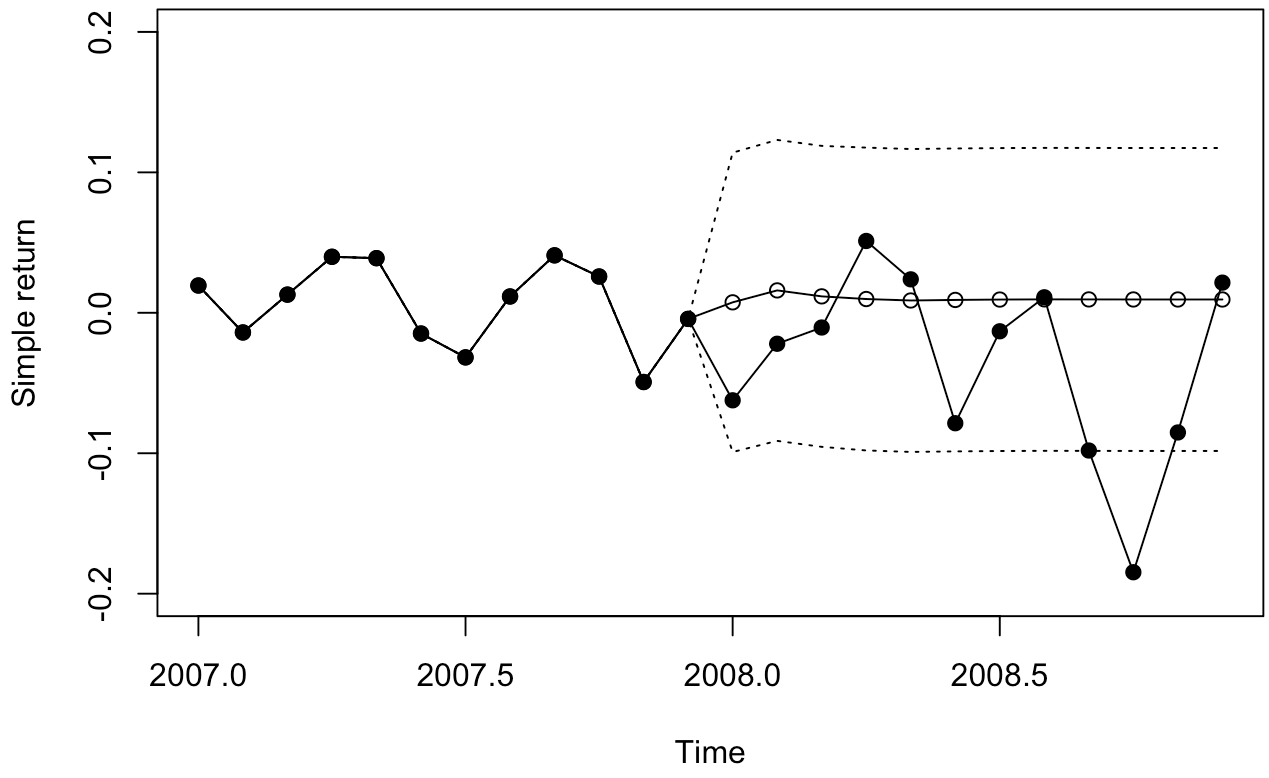

```
## prediction, origin: December 2007, indexed by t=984
vw.ar3=arima(vw[1:984],order=c(3,0,0))
(1-sum(vw.ar3$coef[1:3]))*vw.ar3$coef[4]
```

```
##   intercept
## 0.009728423
```

```
vw.ar3
```

```
##
## Call:
## arima(x = vw[1:984], order = c(3, 0, 0))
##
## Coefficients:
##          ar1      ar2      ar3  intercept
##       0.1034  -0.0201  -0.1089     0.0095
## s.e.  0.0317   0.0318   0.0317     0.0017
##
## sigma^2 estimated as 0.00284:  log likelihood = 1488.85,  aic = -2967.71
```

```
vw.ar3.pred=predict(vw.ar3,12)
vw.ts=ts(vw[973:996],frequency=12,start=c(2007,1))
plot(vw.ts,ylim=c(-.2,.2),ylab="Simple return")
points(vw.ts,pch=19)
vw.pred=ts(c(vw[973:984],vw.ar3.pred$pred),frequency=12,start=c(2007,1))
vw.pred.up=ts(c(vw[973:984],vw.ar3.pred$pred+2*vw.ar3.pred$se),frequency=12,start=c(200
7,1))
vw.pred.lo=ts(c(vw[973:984],vw.ar3.pred$pred-2*vw.ar3.pred$se),frequency=12,start=c(200
7,1))
lines(vw.pred)
points(vw.pred,pch=21)
lines(vw.pred.up,lty=3)
lines(vw.pred.lo,lty=3)
```




```
## a better AR model
vw.ar9=arima(vw,order=c(9,0,0))
vw.ar9
```

```
##
## Call:
## arima(x = vw, order = c(9, 0, 0))
##
## Coefficients:
##          ar1      ar2      ar3     ar4     ar5      ar6     ar7     ar8     ar9
##       0.1167  -0.0111  -0.1128  0.0216  0.0735  -0.0452  0.0254  0.0462  0.0658
## s.e.  0.0316   0.0318   0.0320  0.0322  0.0321   0.0322  0.0320  0.0320  0.0319
##       intercept
##          0.0089
## s.e.     0.0021
##
## sigma^2 estimated as 0.002831:  log likelihood = 1508.54,  aic = -2995.07
```

```
(1-sum(vw.ar9$coef[1:9]))*vw.ar9$coef[10]
```

```
##   intercept
## 0.007278424
```

```
aic=vw.ar9$aic
Box.test(vw.ar9$residuals,lag=18,type='Ljung')
```

```
##
##  Box-Ljung test
##
## data:  vw.ar9$residuals
## X-squared = 17.1, df = 18, p-value = 0.5162
```

```
pv=1-pchisq(17.1,9) # Compute p-value using 3 degrees of freedom
pv
```

```
## [1] 0.04717282
```

```
## To fix the some coefficients to zero, the subcommand "fixed" is used to fix parameter
values, where NA denotes estimation and 0 means fixing the parameter to 0. The ordering
of the parameters can be found using vw.ar3.2null$coef.
vw.ar9.1=arima(vw,order=c(9,0,0),fixed=c(NA,0,NA,0,NA,0,0,NA,NA,NA))
```

```
## Warning in arima(vw, order = c(9, 0, 0), fixed = c(NA, 0, NA, 0, NA, 0, : some
## AR parameters were fixed: setting transform.pars = FALSE
```

```
vw.ar9.1
```

```
##
## Call:
## arima(x = vw, order = c(9, 0, 0), fixed = c(NA, 0, NA, 0, NA, 0, 0, NA, NA,
##     NA))
##
## Coefficients:
##          ar1  ar2      ar3  ar4     ar5  ar6  ar7     ar8     ar9  intercept
##       0.1092    0  -0.1070    0  0.0722    0    0  0.0491  0.0718     0.0088
## s.e.  0.0311    0   0.0314    0  0.0316    0    0  0.0318  0.0316     0.0021
##
## sigma^2 estimated as 0.002839:  log likelihood = 1507.11,  aic = -3000.23
```

```
aic=c(aic,vw.ar9.1$aic)
aic
```

```
## [1] -2995.074 -3000.226
```

```
sqrt(vw.ar9.1$sigma2) # compute residual standard error
```

```
## [1] 0.05328104
```

```
Box.test(vw.ar9.1$residuals,lag=18,type='Ljung')
```

```
##
##  Box-Ljung test
##
## data:  vw.ar9.1$residuals
## X-squared = 19.415, df = 18, p-value = 0.3667
```

```
pv=1-pchisq(19.415,13)
pv
```

```
## [1] 0.1107878
```

2. Estimation: MA Models.

```
vw.ma9=arima(vw,order=c(0,0,9))
vw.ma9
```

```
##
## Call:
## arima(x = vw, order = c(0, 0, 9))
##
## Coefficients:
##          ma1     ma2      ma3     ma4     ma5      ma6     ma7     ma8     ma9
##       0.1151  0.0033  -0.1175  0.0006  0.0871  -0.0087  0.0253  0.0263  0.0904
## s.e.  0.0315  0.0317   0.0320  0.0326  0.0318   0.0320  0.0337  0.0334  0.0314
##       intercept
##          0.0089
## s.e.     0.0021
##
## sigma^2 estimated as 0.002827:  log likelihood = 1509.18,  aic = -2996.37
```

```
aic=c(aic,vw.ma9$aic)
Box.test(vw.ma9$residuals,lag=18,type="Ljung")
```

```
##
##  Box-Ljung test
##
## data:  vw.ma9$residuals
## X-squared = 15.222, df = 18, p-value = 0.6467
```

```
pv=1-pchisq(15.222,9) # Compute p-value using 11 degrees of freedom
pv
```

```
## [1] 0.08501575
```

```
vw.ma9.3=arima(vw,order=c(0,0,9),fixed=c(NA,0,NA,0,NA,0,0,0,NA,NA))
vw.ma9.3
```

```
##
## Call:
## arima(x = vw, order = c(0, 0, 9), fixed = c(NA, 0, NA, 0, NA, 0, 0, 0, NA, NA))
##
## Coefficients:
##          ma1  ma2      ma3  ma4     ma5  ma6  ma7  ma8     ma9  intercept
##       0.1167    0  -0.1215    0  0.0905    0    0    0  0.0860     0.0089
## s.e.  0.0310    0   0.0318    0  0.0308    0    0    0  0.0305     0.0020
##
## sigma^2 estimated as 0.002831:  log likelihood = 1508.52,  aic = -3005.03
```

```
aic=c(aic,vw.ma9.3$aic)
aic
```

```
## [1] -2995.074 -3000.226 -2996.367 -3005.032
```

```
Box.test(vw.ma9.3$residuals,lag=18,type='Ljung')
```

```
##
##  Box-Ljung test
##
## data:  vw.ma9.3$residuals
## X-squared = 17.107, df = 18, p-value = 0.5157
```

```
pv=1-pchisq(17.107,14) # Compute p-value using 9 degrees of freedom
pv
```

```
## [1] 0.2505207
```

3. Estimation: ARMA Models.

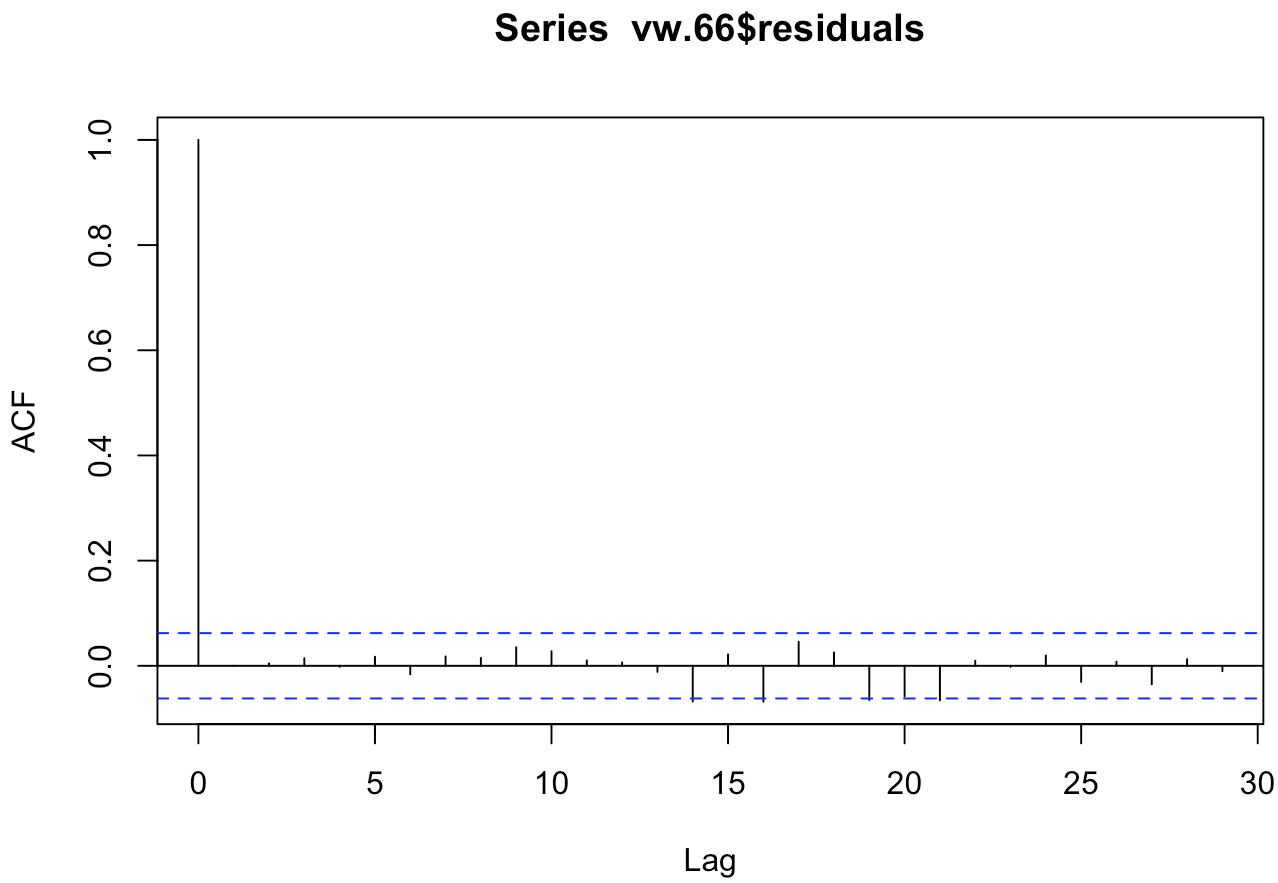
```
vw.66=arima(vw,order=c(6,0,6))
```

```
## Warning in arima(vw, order = c(6, 0, 6)): possible convergence problem: optim
## gave code = 1
```

```
vw.66
```

```
##
## Call:
## arima(x = vw, order = c(6, 0, 6))
##
## Coefficients:
##           ar1     ar2     ar3      ar4      ar5      ar6     ma1      ma2
##       -0.1415  0.0654  0.0196  -0.5211  -0.1107  -0.6145  0.2567  -0.0596
## s.e.   0.1728  0.0927  0.1226   0.0669   0.1684   0.0913  0.1723   0.0919
##           ma3     ma4     ma5     ma6  intercept
##       -0.1573  0.5205  0.2522  0.6635     0.0089
## s.e.   0.1245  0.0666  0.1706  0.0894     0.0018
##
## sigma^2 estimated as 0.002758:  log likelihood = 1517.09,  aic = -3006.19
```

```
aic=c(aic,vw.66$aic)
acf(vw.66$residuals)
```




```
Box.test(vw.66$residuals,lag=18,type="Ljung")
```

```
##
##  Box-Ljung test
##
## data:  vw.66$residuals
## X-squared = 16.325, df = 18, p-value = 0.5699
```

```
pv=1-pchisq(16.325,6) # Compute p-value using 11 degrees of freedom
pv
```

```
## [1] 0.01211189
```

```
vw.66.1=arima(vw,order=c(6,0,6),fixed=c(NA,0,NA,NA,0,NA,NA,0,NA,NA,NA,NA,NA))
```

```
## Warning in arima(vw, order = c(6, 0, 6), fixed = c(NA, 0, NA, NA, 0, NA, : some
## AR parameters were fixed: setting transform.pars = FALSE
```

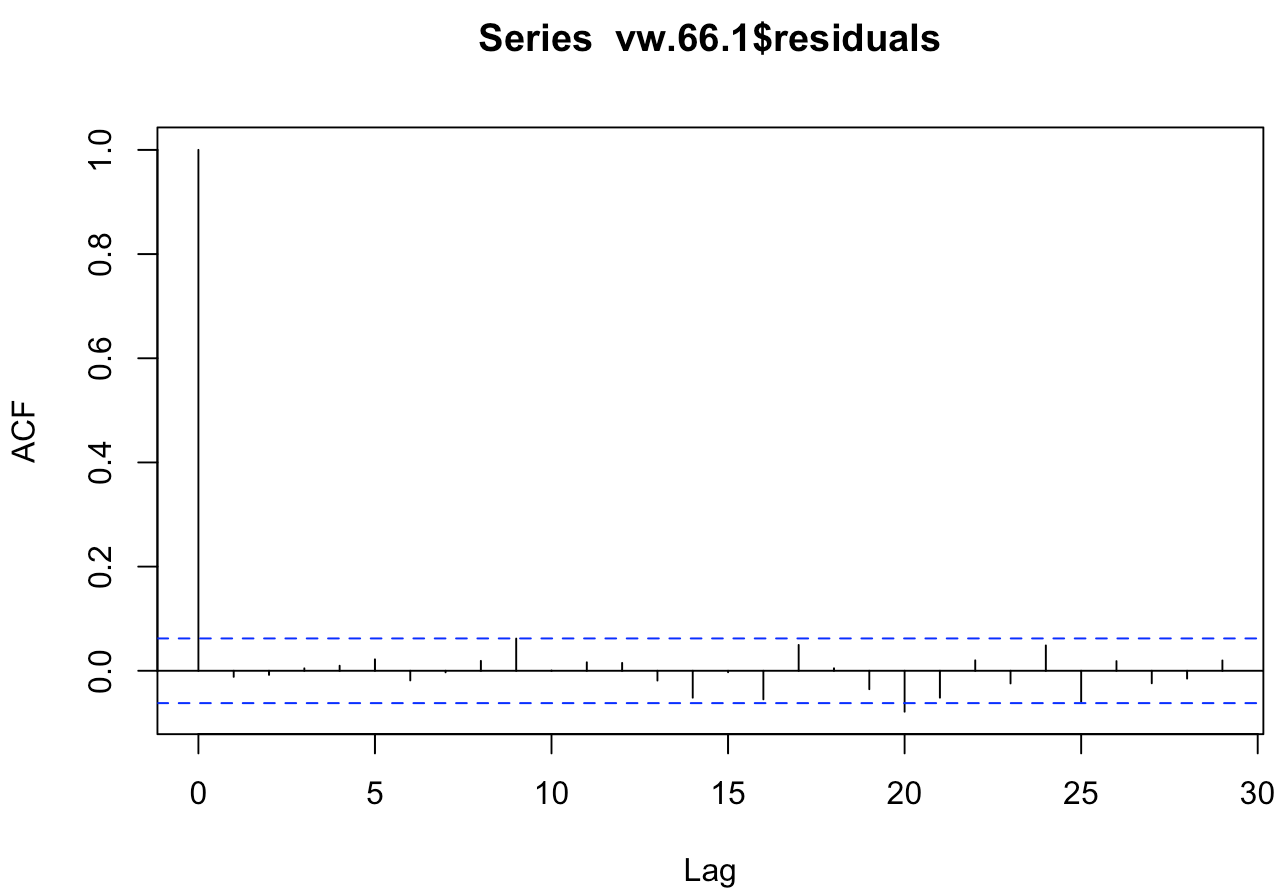
```
vw.66.1
```

```
##
## Call:
## arima(x = vw, order = c(6, 0, 6), fixed = c(NA, 0, NA, NA, 0, NA, NA, 0, NA,
##     NA, NA, NA, NA))
##
## Coefficients:
##          ar1  ar2     ar3      ar4  ar5      ar6     ma1  ma2      ma3     ma4
##       0.0381    0  0.1827  -0.7195    0  -0.4706  0.0905    0  -0.3069  0.7139
## s.e.  0.0137    0  0.0132   0.0129    0   0.0135  0.0314    0   0.0159  0.0127
##          ma5     ma6  intercept
##       0.1523  0.5211     0.0089
## s.e.  0.0291  0.0126     0.0018
##
## sigma^2 estimated as 0.002754:  log likelihood = 1517.55,  aic = -3013.09
```

```
aic=c(aic,vw.66.1$aic)
aic
```

```
## [1] -2995.074 -3000.226 -2996.367 -3005.032 -3006.189 -3013.093
```

```
acf(vw.66.1$residuals)
```



```
Box.test(vw.66.1$residuals,lag=18,type="Ljung")
```

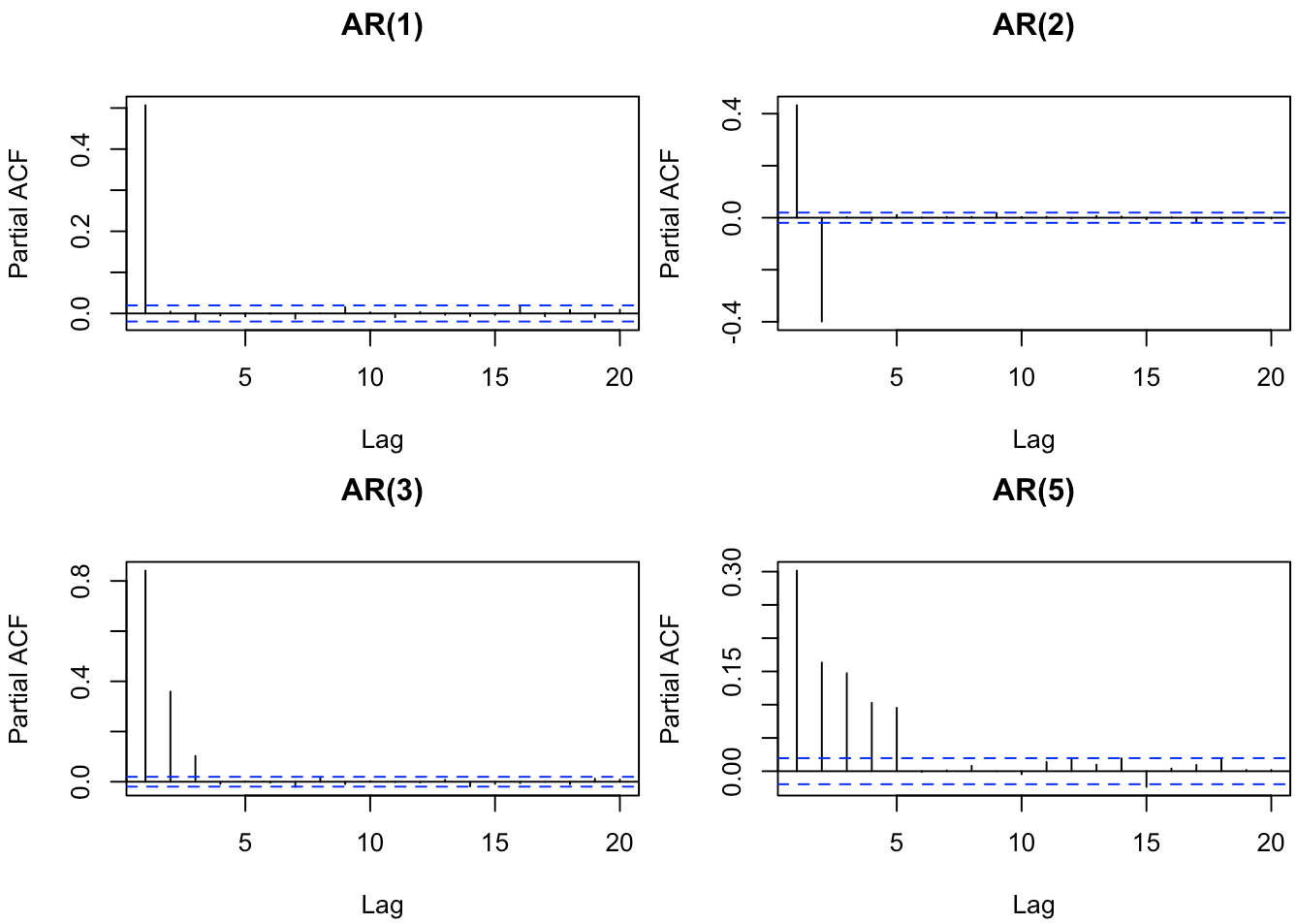
```
##
##  Box-Ljung test
##
## data:  vw.66.1$residuals
## X-squared = 14.472, df = 18, p-value = 0.6979
```

```
pv=1-pchisq(14.472,9) # Compute p-value using 11 degrees of freedom
pv
```

```
## [1] 0.1064981
```

4. Order determinatin for AR models: PACF.

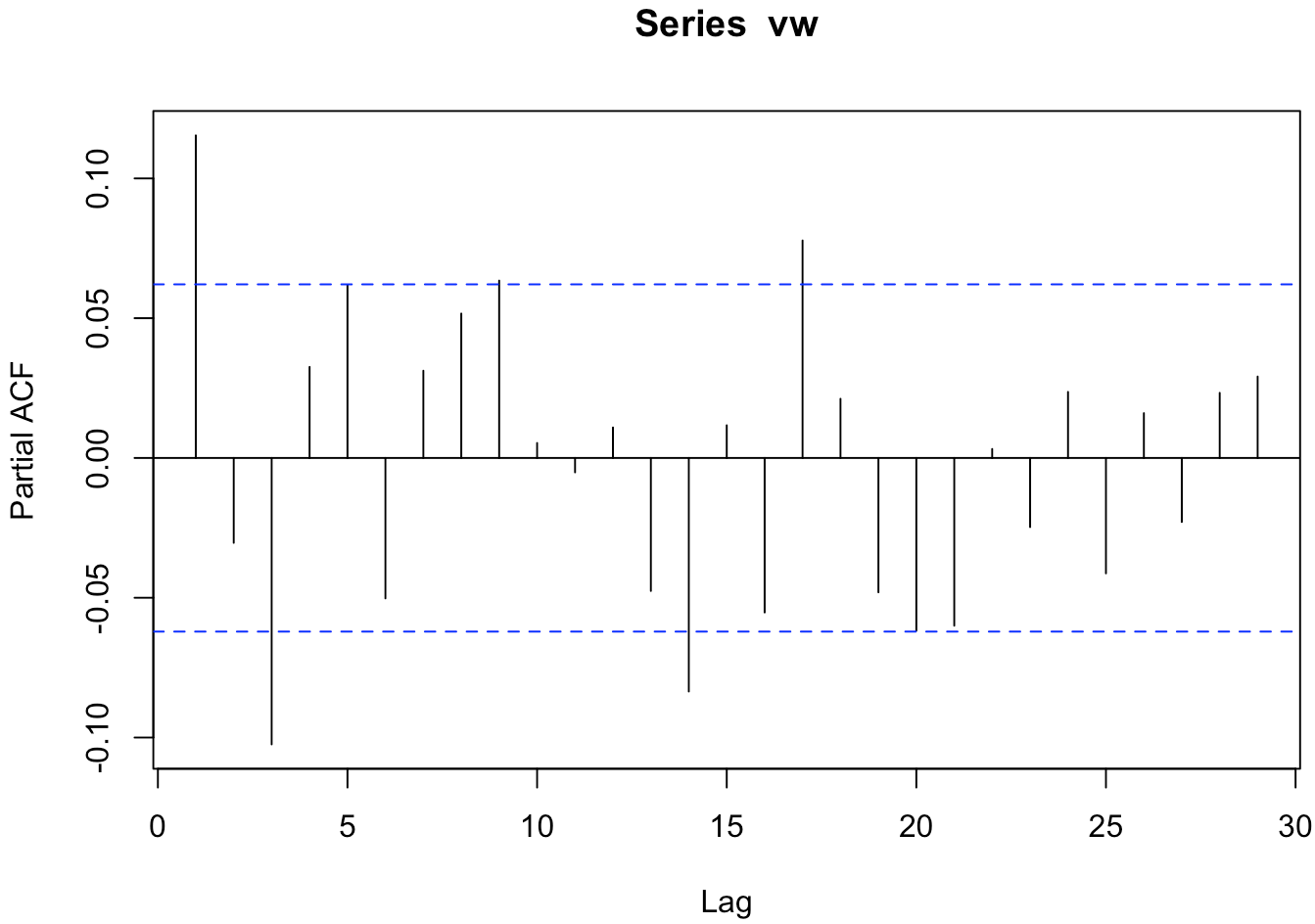
```
###################### PACF for various AR processes
T=10000
y0=arima.sim(model=list(ar=c(.5)),T)
y1=arima.sim(model=list(ar=c(1.2,-.35)),T)
y2=arima.sim(model=list(ar=c(.6,-.4)),T)
y3=arima.sim(model=list(ar=c(.2,.35)),T)
y4=arima.sim(model=list(ar=c(-.2,.35)),T)
y5=arima.sim(model=list(ar=c(.5,.3,.1)),T)
y6=arima.sim(model=list(ar=c(.2,.1,.1,.1,.1)),T)
par(mfrow=c(2,2))
par(mar=c(4,4,3.5,.5))
acf(y0,type="partial",main="AR(1)",20)
acf(y2,type="partial",main="AR(2)",20)
acf(y5,type="partial",main="AR(3)",20)
acf(y6,type="partial",main="AR(5)",20)
```




```
vw.ord=ar(vw,method="mle")
vw.ord$order
```

```
## [1] 9
```

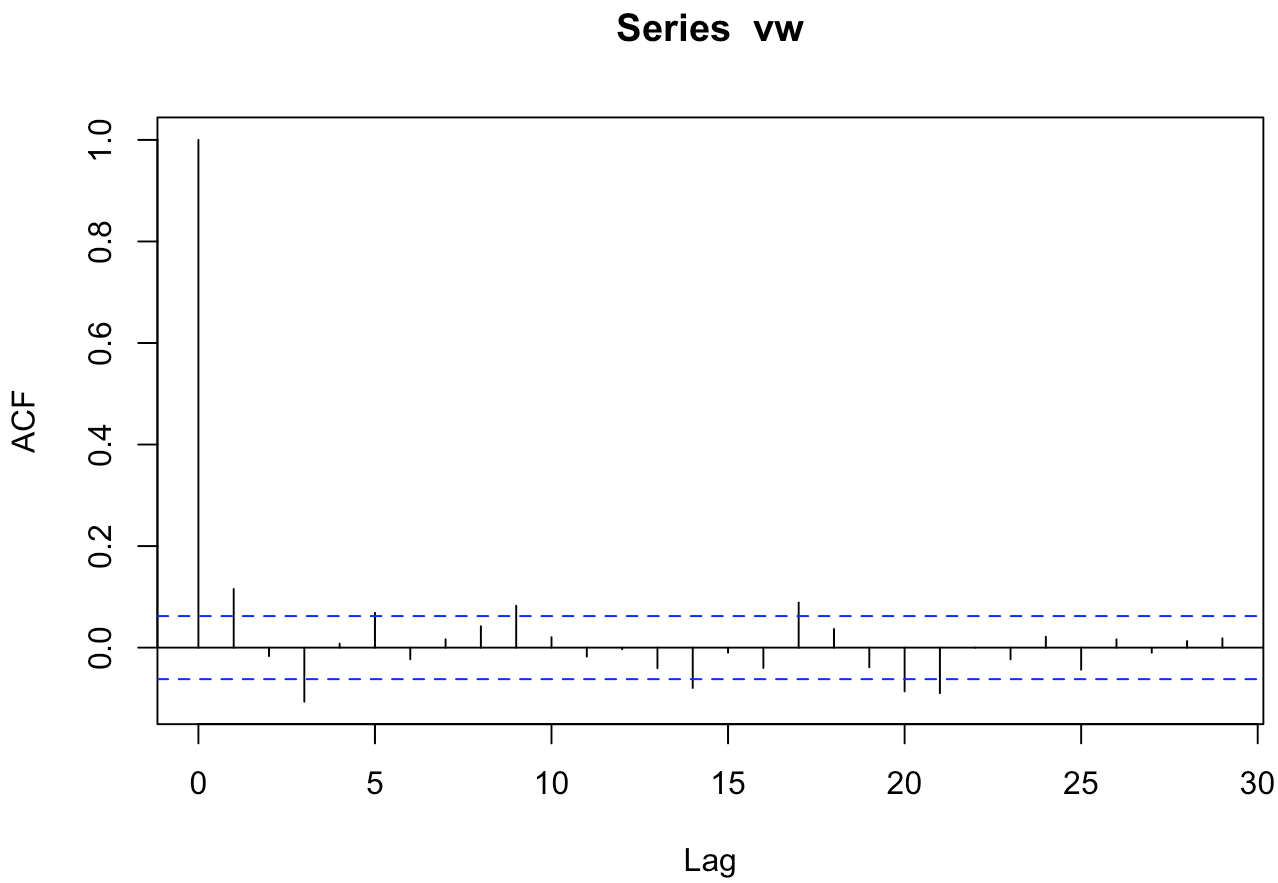
```
par(mfrow=c(1,1))
pacf(vw)
```



5. Order determination for MA models: ACF.

Only need to pay attention to the plot for now!

```
acf(vw)
```



6. Order determination for ARMA models: AIC and EACF.

```
pqmax=7
aic=array(0,dim=c(pqmax,pqmax))
for (p in 1:pqmax){
    for (q in 1:pqmax){
        vw.arma=arima(vw,order=c(p,0,q))
        aic[p,q]=vw.arma$aic
    }
}
```

```
## Warning in arima(vw, order = c(p, 0, q)): possible convergence problem: optim
## gave code = 1
## Warning in arima(vw, order = c(p, 0, q)): possible convergence problem: optim
## gave code = 1
## Warning in arima(vw, order = c(p, 0, q)): possible convergence problem: optim
## gave code = 1
## Warning in arima(vw, order = c(p, 0, q)): possible convergence problem: optim
## gave code = 1
```

```
aic
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,] -2982.412 -2980.393 -2991.064 -2989.049 -2993.401 -2991.399 -2991.858
## [2,] -2983.512 -2997.019 -2996.014 -2994.090 -2993.517 -2991.889 -2989.916
## [3,] -2990.149 -2995.982 -2994.061 -2992.067 -2991.907 -2989.935 -2987.944
## [4,] -2991.101 -2994.096 -2992.056 -2990.077 -2990.102 -2988.258 -2989.861
## [5,] -2993.119 -2991.399 -2989.975 -2990.006 -3005.282 -3003.520 -2990.457
## [6,] -2991.474 -2990.486 -2988.036 -2988.008 -3004.018 -3006.189 -3003.373
## [7,] -2993.458 -3000.302 -2999.662 -2993.431 -2998.746 -3003.775 -3004.935
```

```
which(aic==min(aic))
```

```
## [1] 41
```

```
###################### EACF
source("EACF.R")
da=read.table("m-3m4608.txt",header=T)
m3m=da[,2]
EACF(vw,p=12,q=12)$seacf
```

```
## Warning in optim(coef, err, gr = NULL, hessian = TRUE, ...): one-dimensional optimiza
tion by Nelder-Mead is unreliable:
## use "Brent" or optimize() directly
```

```
## [1] "EACF table"
##              [,1]        [,2]         [,3]         [,4]       [,5]
##  [1,]  0.11539600 -0.01664176 -0.106480031  0.007911439 0.06858793
##  [2,]  0.24704638 -0.01784877 -0.105371713  0.011830709 0.06021722
##  [3,] -0.26924556  0.49921318 -0.026167443 -0.019946665 0.07030038
##  [4,]  0.28012192  0.44994318 -0.301207483 -0.011204754 0.05819736
##  [5,] -0.40277333  0.42622891  0.113441228  0.149072694 0.02753412
##  [6,]  0.48874303  0.06899561 -0.400696120 -0.103402181 0.22794687
##  [7,]  0.44498484  0.05552152 -0.179834289 -0.137436698 0.17116972
##  [8,] -0.43282787  0.07673156  0.154202449 -0.262628570 0.02662393
##  [9,] -0.48894271  0.11596858  0.397120204 -0.265306253 0.02175676
## [10,] -0.08631314  0.08041711 -0.186543887  0.288681938 0.13033361
## [11,]  0.48750997 -0.02152969  0.008530498  0.185213831 0.13366614
## [12,]  0.33415477  0.06475110  0.001991203  0.141581509 0.28005871
##                [,6]       [,7]         [,8]        [,9]        [,10]
##  [1,] -0.0228904236 0.01637031  0.042092629 0.082384736  0.020450750
##  [2,]  0.0156419405 0.01077242  0.002439166 0.071341655  0.046868031
##  [3,] -0.0119656004 0.01891062 -0.003198579 0.062690433  0.032236448
##  [4,] -0.0223833639 0.01515773  0.027470811 0.061479506  0.015328568
##  [5,]  0.0162766355 0.07062800  0.014321762 0.077953429 -0.020120690
##  [6,] -0.0173982792 0.06957289 -0.013726814 0.045619597  0.017006559
##  [7,]  0.2688628771 0.03605138 -0.007619239 0.046794896  0.010107783
##  [8,]  0.3473008881 0.06151459 -0.003813628 0.039032314  0.021879488
##  [9,]  0.3287652313 0.02642623 -0.192961563 0.006326578  0.005408022
## [10,] -0.0006014256 0.14127652 -0.219590776 0.160218768 -0.005400353
## [11,] -0.0020217721 0.05394562 -0.215603856 0.049157204  0.053799766
## [12,]  0.0330164404 0.08311817 -0.068477493 0.129912419 -0.058869470
##              [,11]         [,12]
##  [1,] -0.017519210 -0.0030189831
##  [2,] -0.021081935  0.0011231923
##  [3,] -0.020562706 -0.0020012085
##  [4,] -0.024999702  0.0003502985
##  [5,]  0.009868639 -0.0159566883
##  [6,] -0.013940644  0.0010893459
##  [7,] -0.009698022  0.0019768977
##  [8,]  0.007327473  0.0051041722
##  [9,]  0.023572549  0.0004367239
## [10,]  0.007957083  0.0091069664
## [11,]  0.005635660  0.0121847939
## [12,]  0.114446819  0.0002917764
## [1] " "
## [1] "Simplified EACF: 2 denotes significance"
##       [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12]
##  [1,]    2    0    2    0    2    0    0    0    2     0     0     0
##  [2,]    2    0    2    0    0    0    0    0    2     0     0     0
##  [3,]    2    2    0    0    2    0    0    0    0     0     0     0
##  [4,]    2    2    2    0    0    0    0    0    0     0     0     0
##  [5,]    2    2    2    2    0    0    2    0    2     0     0     0
##  [6,]    2    2    2    2    2    0    2    0    0     0     0     0
##  [7,]    2    0    2    2    2    2    0    0    0     0     0     0
##  [8,]    2    2    2    2    0    2    0    0    0     0     0     0
##  [9,]    2    2    2    2    0    2    0    2    0     0     0     0
```

```
## [10,]    2    2    2    2    2    0    2    2    2     0     0     0
## [11,]    2    0    0    2    2    0    0    2    0     0     0     0
## [12,]    2    2    0    2    2    0    2    2    2     0     2     0
```

```
##       [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12]
##  [1,]    2    0    2    0    2    0    0    0    2     0     0     0
##  [2,]    2    0    2    0    0    0    0    0    2     0     0     0
##  [3,]    2    2    0    0    2    0    0    0    0     0     0     0
##  [4,]    2    2    2    0    0    0    0    0    0     0     0     0
##  [5,]    2    2    2    2    0    0    2    0    2     0     0     0
##  [6,]    2    2    2    2    2    0    2    0    0     0     0     0
##  [7,]    2    0    2    2    2    2    0    0    0     0     0     0
##  [8,]    2    2    2    2    0    2    0    0    0     0     0     0
##  [9,]    2    2    2    2    0    2    0    2    0     0     0     0
## [10,]    2    2    2    2    2    0    2    2    2     0     0     0
## [11,]    2    0    0    2    2    0    0    2    0     0     0     0
## [12,]    2    2    0    2    2    0    2    2    2     0     2     0
```

7. Model selection: rolling forecasting.

```
source("rolling.forecast.R")
x=arima.sim(model=list(ar=c(0.7,-0.3)),sd=1,n=3000)
out=arima(x,c(1,0,0))
out
```

```
##
## Call:
## arima(x = x, order = c(1, 0, 0))
##
## Coefficients:
##          ar1  intercept
##       0.5569    -0.0029
## s.e.  0.0152     0.0425
##
## sigma^2 estimated as 1.064:  log likelihood = -4350.14,  aic = 8706.27
```

```
out2=arima(x,c(2,0,0))
out2
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 0))
##
## Coefficients:
##          ar1      ar2  intercept
##       0.7147  -0.2833    -0.0029
## s.e.  0.0175   0.0175     0.0318
##
## sigma^2 estimated as 0.9786:  log likelihood = -4224.56,  aic = 8457.12
```

```
out3=arima(x,c(5,0,0))
out3
```

```
##
## Call:
## arima(x = x, order = c(5, 0, 0))
##
## Coefficients:
##          ar1      ar2     ar3      ar4      ar5  intercept
##       0.7191  -0.2974  0.0210  -0.0055  -0.0077    -0.0029
## s.e.  0.0183   0.0225  0.0231   0.0225   0.0183     0.0316
##
## sigma^2 estimated as 0.9781:  log likelihood = -4223.94,  aic = 8461.87
```

```
print(rolling.forecast(x,5,2500,c(1,0,0)))
```

```
## [1] 528.3072 811.5613 860.7018 862.5279 854.3488
```

```
print(rolling.forecast(x,5,2500,c(2,0,0)))
```

```
## [1] 496.3977 804.6388 849.9682 844.3166 846.6713
```

```
print(rolling.forecast(x,5,2500,c(5,0,0)))
```

```
## [1] 496.5257 804.3696 848.7184 843.5296 845.9117
```

8. Diagnostics.

```
## ARMA estimation
set.seed(101)
x=arima.sim(model=list(ar=c(0,0.9),ma=c(0.7,0.5)),sd=1,n=200)
# order specification
out=arima(x,c(2,0,2))
out
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 2))
##
## Coefficients:
##          ar1     ar2     ma1     ma2  intercept
##       0.0109  0.8800  0.6412  0.4853    -0.5178
## s.e.  0.0335  0.0341  0.0709  0.0823     1.2010
##
## sigma^2 estimated as 0.8874:  log likelihood = -274.23,  aic = 560.45
```

```
## ARIMA estimation: (a redundant model)
out=arima(x,c(2,0,2),include.mean=F)
out
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 2), include.mean = F)
##
## Coefficients:
##          ar1     ar2     ma1     ma2
##       0.0124  0.8816  0.6411  0.4851
## s.e.  0.0332  0.0338  0.0709  0.0824
##
## sigma^2 estimated as 0.8881:  log likelihood = -274.32,  aic = 558.63
```

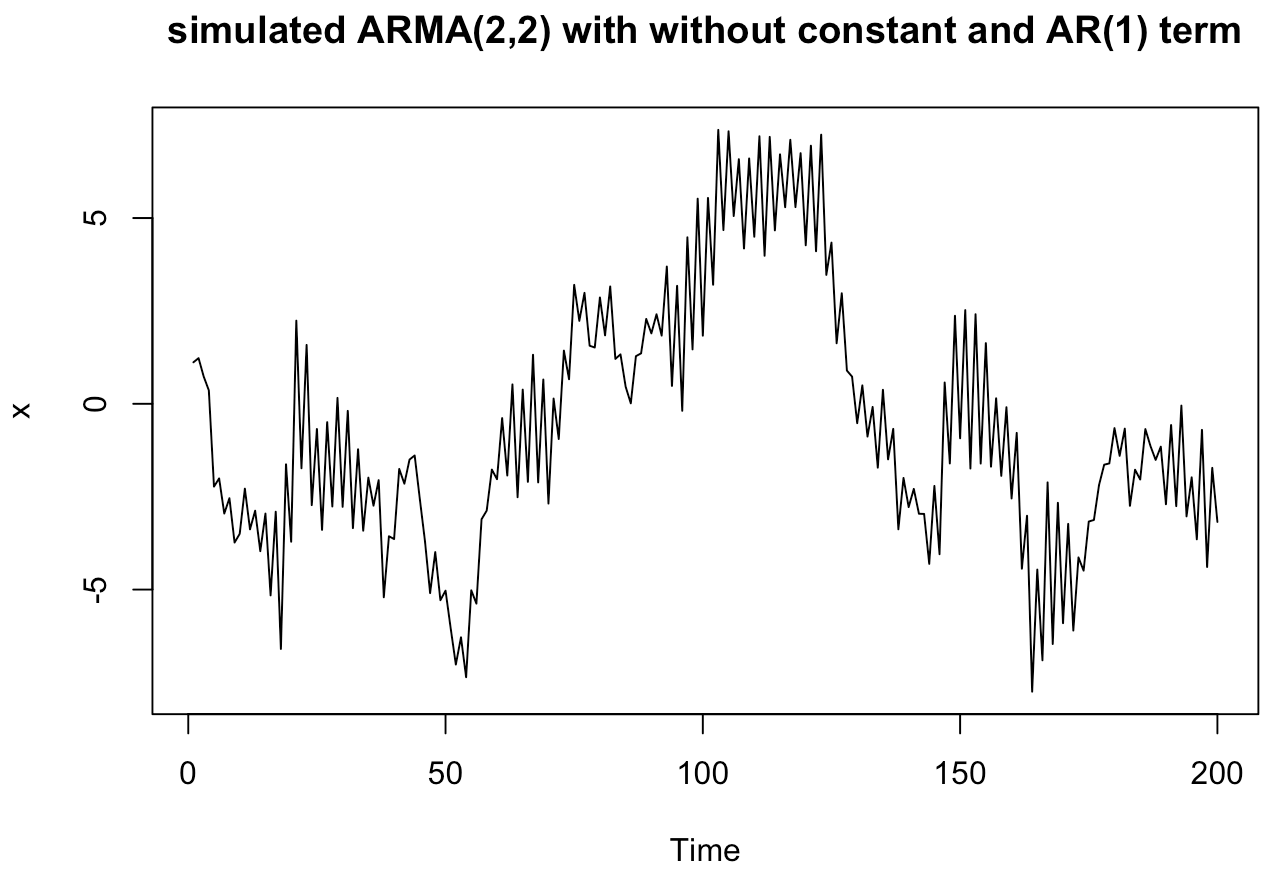
```
# fixed some coefficients
out=arima(x,order=c(2,0,2),fixed=c(0,NA,NA,NA),include.mean=F)
```

```
## Warning in arima(x, order = c(2, 0, 2), fixed = c(0, NA, NA, NA), include.mean
## = F): some AR parameters were fixed: setting transform.pars = FALSE
```

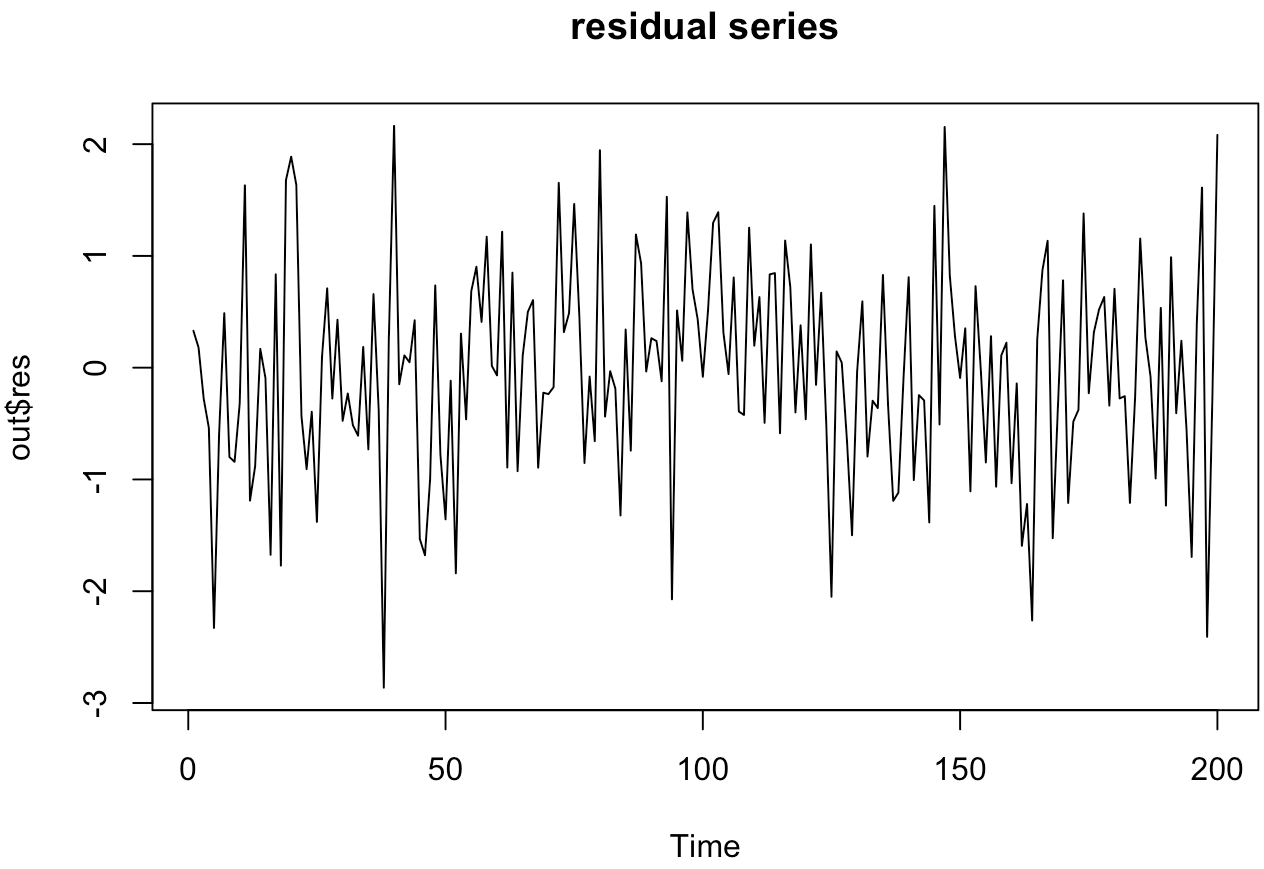
```
out
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 2), include.mean = F, fixed = c(0, NA, NA, NA))
##
## Coefficients:
##       ar1     ar2     ma1     ma2
##         0  0.8834  0.6485  0.4860
## s.e.    0  0.0333  0.0674  0.0826
##
## sigma^2 estimated as 0.8886:  log likelihood = -274.39,  aic = 556.77
```

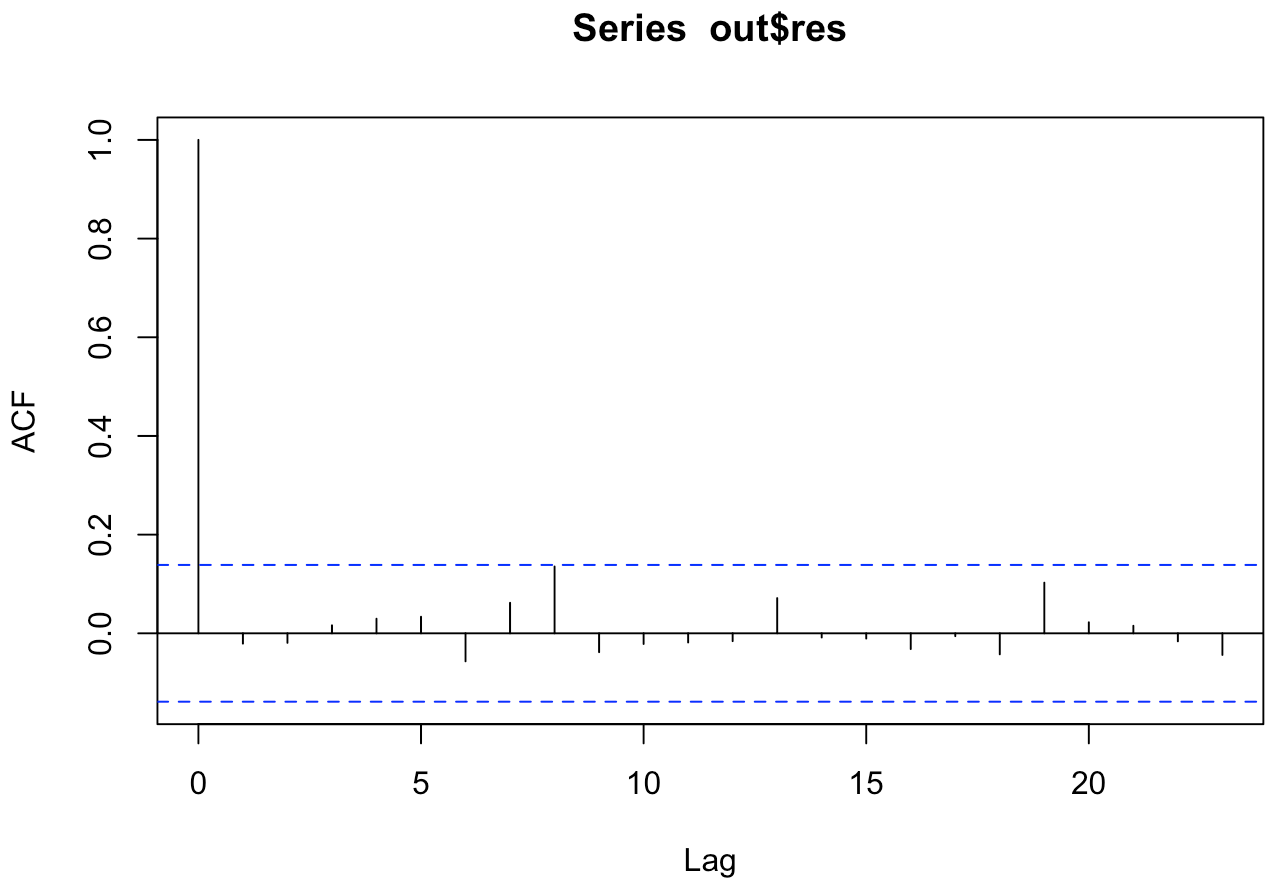
```
plot(x)
title("simulated ARMA(2,2) with without constant and AR(1) term")
```




```
plot(out$res)
title("residual series")
```




```
acf=acf(out$res,na.action=na.pass)
```




```
## ARIMA estimation: (another redudant model)
out=arima(x,c(3,0,2),include.mean=F)
out
```

```
##
## Call:
## arima(x = x, order = c(3, 0, 2), include.mean = F)
##
## Coefficients:
##           ar1     ar2     ar3     ma1     ma2
##       -0.1130  0.8775  0.1261  0.7428  0.5525
## s.e.   0.1314  0.0346  0.1287  0.1137  0.0983
##
## sigma^2 estimated as 0.884:  log likelihood = -273.89,  aic = 559.78
```

```
## roots
polyroot(c(1,-out$coef[1:3]))    ## for AR roots, use negative coef
```

```
## [1]  1.052402-3.231174e-27i -1.088470-1.704654e-20i -6.922770+1.704654e-20i
```

```
abs(polyroot(c(1,-out$coef[1:3])))
```

```
## [1] 1.052402 1.088470 6.922770
```

```
polyroot(c(1,out$coef[4:5]))
```

```
## [1] -0.6721299+1.165362i -0.6721299-1.165362i
```

```
abs(polyroot(c(1,out$coef[4:5])))
```

```
## [1] 1.345299 1.345299
```

```
## Another Type of Redundancy
set.seed(123)
x=arima.sim(model=list(ar=c(0.8),ma=0.5),sd=1,n=600)
out=arima(x,c(2,0,2),include.mean=F)
```

```
## Warning in arima(x, c(2, 0, 2), include.mean = F): possible convergence
## problem: optim gave code = 1
```

```
out
```

```
##
## Call:
## arima(x = x, order = c(2, 0, 2), include.mean = F)
##
## Coefficients:
##          ar1     ar2     ma1     ma2
##       0.0049  0.5498  1.2346  0.3566
## s.e.  0.4286  0.3226  0.4251  0.2017
##
## sigma^2 estimated as 0.9357:  log likelihood = -832.3,  aic = 1674.59
```

```
polyroot(c(1,-out$coef[1:2]))
```

```
## [1]  1.344167+0i -1.353145+0i
```

```
polyroot(c(1,out$coef[3:4]))
```

```
## [1] -1.292494+5.04871e-29i -2.169540-5.04871e-29i
```

```
out=arima(x,c(1,0,1),include.mean=F)
out
```

```
##
## Call:
## arima(x = x, order = c(1, 0, 1), include.mean = F)
##
## Coefficients:
##          ar1     ma1
##       0.7399  0.4959
## s.e.  0.0305  0.0400
##
## sigma^2 estimated as 0.9363:  log likelihood = -832.45,  aic = 1670.91
```